

Laporan Tugas Aktivitas Mandiri 1 dan 2



Oleh

Desni Eria Purba

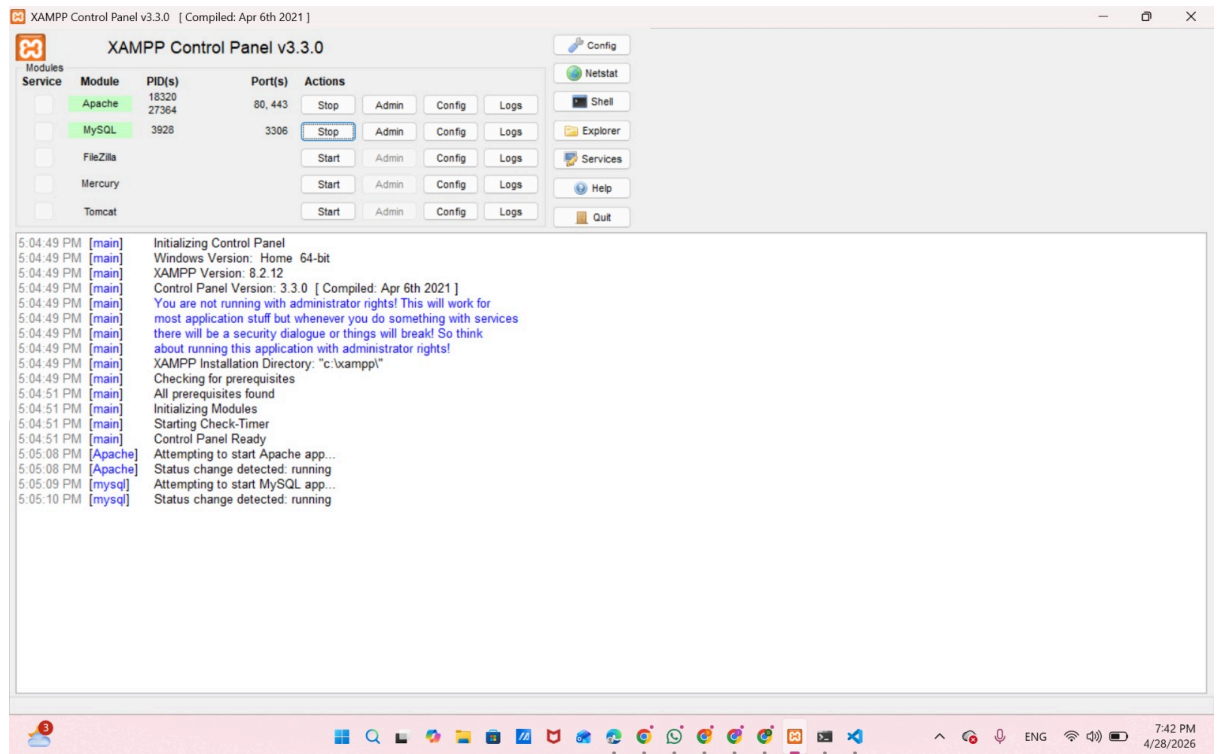
(242410101055)

KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS JEMBER
JEMBER
2025/2026

Laporan Pemrograman Website

Aktivitas Mandiri 1

1. Instal XAMPP terbaru, pastikan PHP 8.2+ tersedia (cek: `php --version` di terminal)



Pada tahap awal, saya menginstal XAMPP sebagai lingkungan server lokal yang digunakan untuk menjalankan Apache dan MySQL. Setelah proses instalasi selesai, saya mengaktifkan layanan Apache dan MySQL melalui XAMPP Control Panel agar server dapat berjalan dengan baik.

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Desni> php --version
PHP 8.3.30 (cli) (built: Jan 13 2026 22:50:40) (ZTS Visual C++ 2019
x64)
Copyright (c) The PHP Group
Zend Engine v4.3.30, Copyright (c) Zend Technologies
```

Selanjutnya, saya melakukan pengecekan versi PHP melalui Command Prompt menggunakan perintah `php --version`. Dari hasil tersebut terlihat bahwa PHP yang digunakan sudah versi 8.3, sehingga sudah memenuhi kebutuhan untuk menjalankan Laravel versi terbaru.

2. Instal Composer dari getcomposer.org, verifikasi dengan: `composer --version`

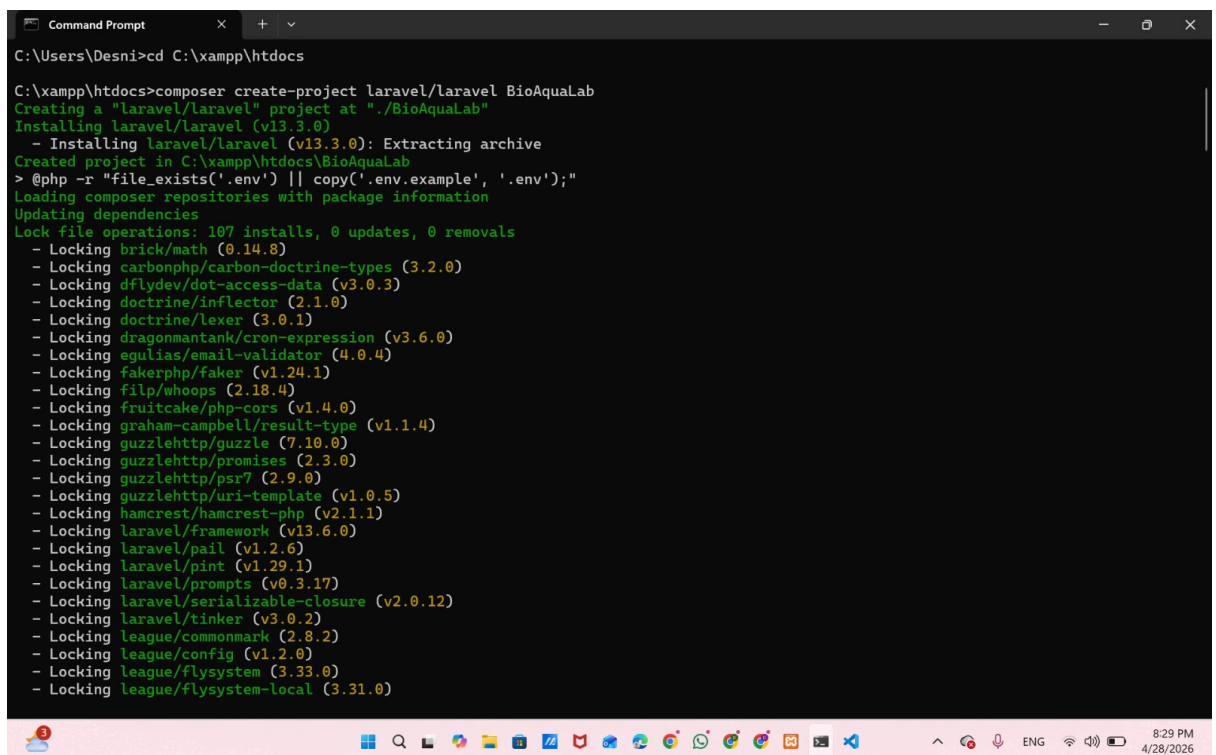
```
C:\Users\Desni> composer --version
Composer version 2.9.7 2026-04-14 13:31:52
PHP version 8.3.30 (C:\laragon\bin\php\php-8.3.30-Win32-vs16-x64\php.exe)
Run the "diagnose" command to get more detailed diagnostics output.

C:\Users\Desni>cd C:\xampp\htdocs
```

Setelah PHP siap digunakan, langkah berikutnya adalah menginstal Composer sebagai dependency manager untuk PHP. Instalasi dilakukan melalui situs resminya, kemudian diverifikasi dengan perintah `composer --version`.

Hasil pengecekan menunjukkan bahwa Composer telah berhasil terpasang dan dapat digunakan bersama PHP yang sudah diinstal sebelumnya.

3. Buat proyek Laravel baru: `composer create-project laravel/laravel siak-app`



```
Command Prompt
C:\Users\Desni>cd C:\xampp\htdocs

C:\xampp\htdocs>composer create-project laravel/laravel BioAquaLab
Creating a "laravel/laravel" project at "./BioAquaLab"
Installing laravel/laravel (v13.3.0)
- Installing laravel/laravel (v13.3.0): Extracting archive
Created project in C:\xampp\htdocs\BioAquaLab
> @php -r "file_exists('.env') || copy('.env.example', '.env');"
Loading composer repositories with package information
Updating dependencies
Lock file operations: 107 installs, 0 updates, 0 removals
- Locking brick/math (0.14.0)
- Locking carbonphp/carbon-doctrine-types (3.2.0)
- Locking dflydev/dot-access-data (v3.0.3)
- Locking doctrine/inflector (2.1.0)
- Locking doctrine/lexer (3.0.1)
- Locking dragonmantank/cron-expression (v3.6.0)
- Locking egulias/email-validator (4.0.4)
- Locking fakerphp/faker (v1.24.1)
- Locking filp/whoops (2.18.4)
- Locking fruitcake/php-cors (v1.4.0)
- Locking graham-campbell/result-type (v1.1.4)
- Locking guzzlehttp/guzzle (7.10.0)
- Locking guzzlehttp/promises (2.3.0)
- Locking guzzlehttp/psr7 (2.9.0)
- Locking guzzlehttp/uri-template (v1.0.5)
- Locking hamcrest/hamcrest-php (v2.1.1)
- Locking laravel/framework (v13.6.0)
- Locking laravel/pail (v1.2.6)
- Locking laravel/pint (v1.29.1)
- Locking laravel/prompts (v0.3.17)
- Locking laravel/serializable-closure (v2.0.12)
- Locking laravel/tinker (v3.0.2)
- Locking league/commonmark (2.8.2)
- Locking league/config (v1.2.0)
- Locking league/flysystem (3.33.0)
- Locking league/flysystem-local (3.31.0)
```

```
Command Prompt
- Locking league/mime-type-detection (1.16.0)
- Locking league/uri (7.8.1)
- Locking league/uri-interfaces (7.8.1)
- Locking mockery/mockery (1.6.12)
- Locking monolog/monolog (3.10.0)
- Locking myclabs/deep-copy (1.13.4)
- Locking nesbot/carbon (3.11.4)
- Locking nette/schema (v1.3.5)
- Locking nette/urls (v4.1.3)
- Locking nikic/php-parser (v5.7.0)
- Locking nunomaduro/collision (v8.9.4)
- Locking nunomaduro/termwind (v2.4.0)
- Locking phar-io/manifest (2.0.4)
- Locking phar-io/version (3.2.1)
- Locking phpoption/phpoption (1.9.5)
- Locking phpunit/php-code-coverage (12.5.6)
- Locking phpunit/php-file-iterator (6.0.1)
- Locking phpunit/php-invoker (6.0.0)
- Locking phpunit/php-text-template (5.0.0)
- Locking phpunit/php-timer (8.0.0)
- Locking phpunit/phpunit (12.5.23)
- Locking psr/clock (1.0.0)
- Locking psr/container (2.0.2)
- Locking psr/event-dispatcher (1.0.0)
- Locking psr/http-client (1.0.3)
- Locking psr/http-factory (1.1.0)
- Locking psr/http-message (2.0)
- Locking psr/log (3.0.2)
- Locking psr/simple-cache (3.0.0)
- Locking psy/psysh (v0.12.22)
- Locking ralouphie/getallheaders (3.0.3)
- Locking ramsey/collection (2.1.1)
- Locking ramsey/uuid (4.9.2)
- Locking sebastian/cli-parser (4.2.0)
- Locking sebastian/comparator (7.1.6)
- Locking sebastian/complextity (5.0.0)
- Locking sebastian/diff (7.0.0)
```

```
Command Prompt
60 package suggestions were added by new dependencies, use 'composer suggest' to see details.
Generating optimized autoload files
> illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

INFO Discovering packages.

laravel/pail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE

78 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
> @php artisan vendor:publish --tag=laravel-assets --ansi --force

INFO No publishable resources for tag [laravel-assets].

No security vulnerability advisories found.
> @php artisan key:generate --ansi

INFO Application key set successfully.

> @php -r "file_exists('database/database.sqlite') || touch('database/database.sqlite');"
> @php artisan migrate --graceful --ansi

INFO Preparing database.

Creating migration table ..... 13.25ms DONE

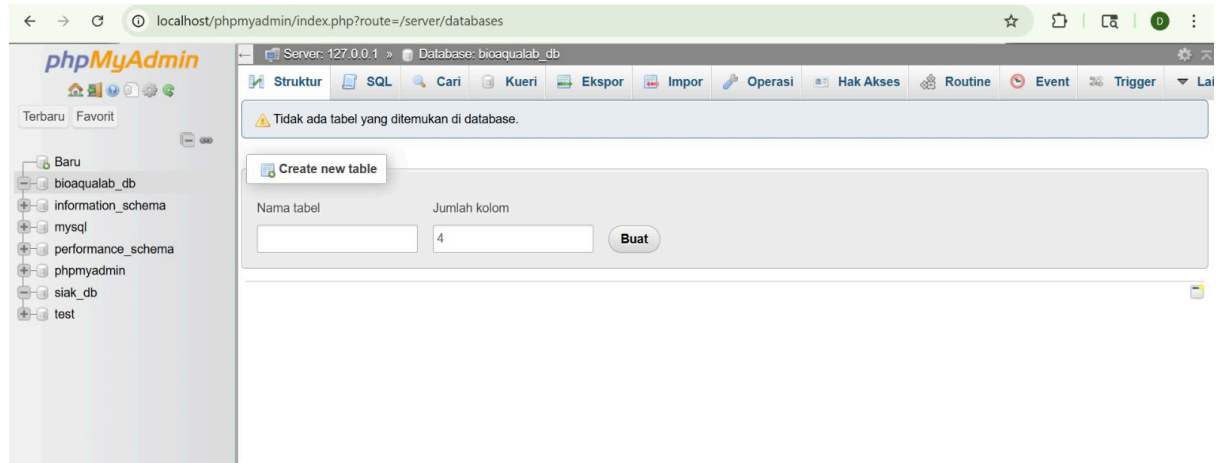
INFO Running migrations.

0001_01_01_000000_create_users_table ..... 29.78ms DONE
0001_01_01_000001_create_cache_table ..... 17.10ms DONE
0001_01_01_000002_create_jobs_table ..... 23.81ms DONE
```

Saya membuat project Laravel baru dengan menjalankan perintah composer create-project laravel/laravel tugas-laravel.

Pada proses ini, Composer secara otomatis mengunduh framework Laravel beserta seluruh library yang dibutuhkan. Setelah selesai, sistem juga langsung membuat application key yang berfungsi untuk keamanan aplikasi.

4. Buat database baru 'siak_db' di phpMyAdmin



Untuk menyimpan data aplikasi, saya membuat database baru melalui phpMyAdmin dengan nama sesuai kebutuhan (berdasarkan gambar).

Database ini masih kosong karena nantinya tabel akan dibuat melalui proses migration dari Laravel.

5. Konfigurasi file .env: isi DB_DATABASE, DB_USERNAME, DB_PASSWORD sesuai lokal

```
DB_CONNECTION=mysql
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=bioaqua_db
# DB_USERNAME=root
# DB_PASSWORD=
```

Agar aplikasi dapat terhubung dengan database, saya melakukan pengaturan pada file .env.

Beberapa bagian yang disesuaikan antara lain:

- DB_DATABASE : bioaqua-db

- DB_USERNAME : root
- DB_PASSWORD :

Konfigurasi ini penting agar Laravel bisa mengakses database dengan benar.

6. Jalankan: php artisan key:generate

```
PS C:\xampp\htdocs\BioAquaLab> php artisan key:generate

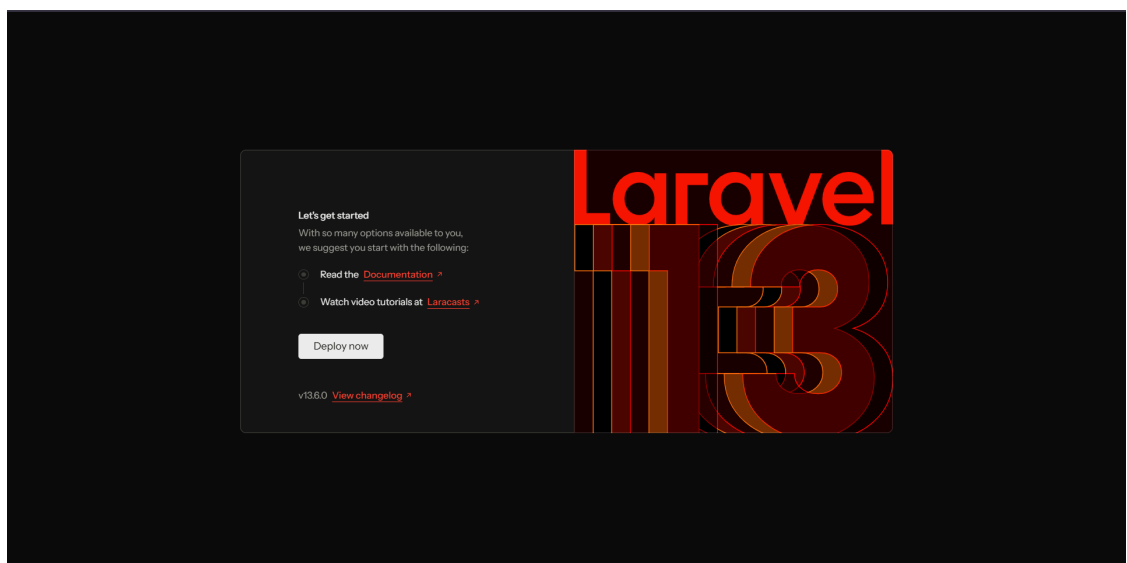
INFO Application key set successfully.
```

Saya menjalankan perintah php artisan key:generate melalui terminal.

Perintah ini digunakan untuk membuat kunci enkripsi yang akan disimpan di file .env.

Kunci ini berfungsi untuk menjaga keamanan data dalam aplikasi.

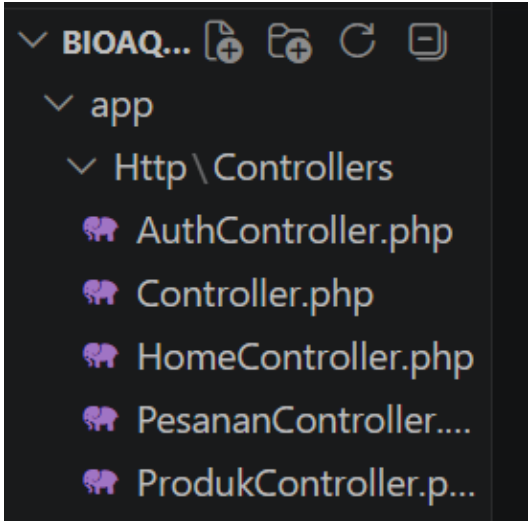
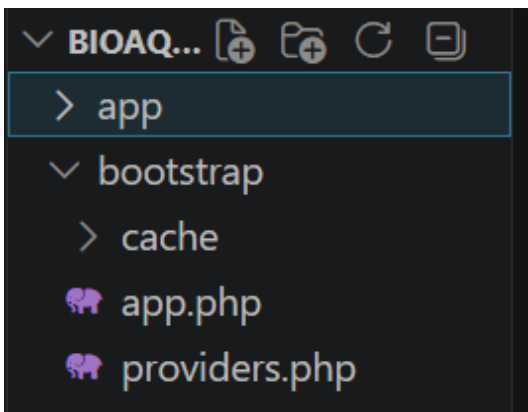
7. Jalankan server: php artisan serve — buka http://127.0.0.1:8000 di browser

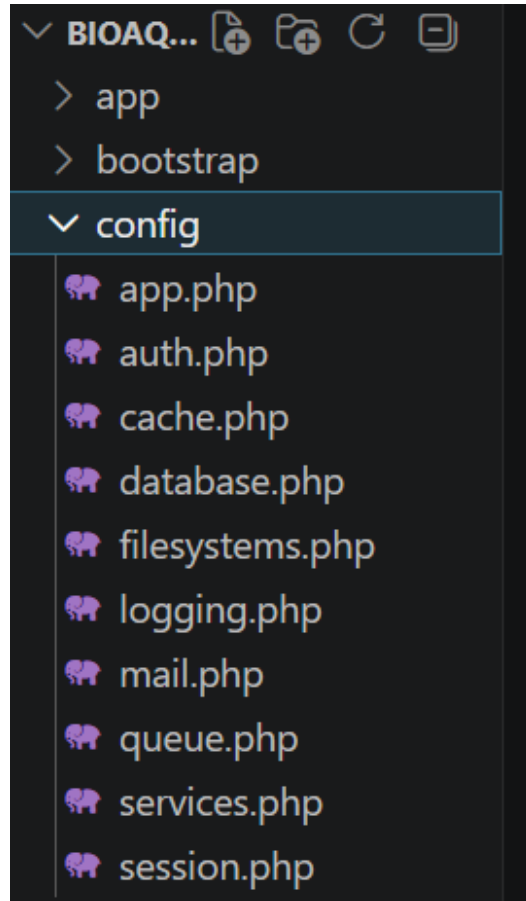


Untuk menjalankan aplikasi, saya menggunakan perintah php artisan serve. Setelah server aktif, aplikasi dapat diakses melalui browser di alamat: <http://127.0.0.1:8000> Jika halaman Laravel muncul, berarti aplikasi sudah berhasil dijalankan.

8. Eksplorasi struktur folder, buka setiap folder dan pahami fungsinya:

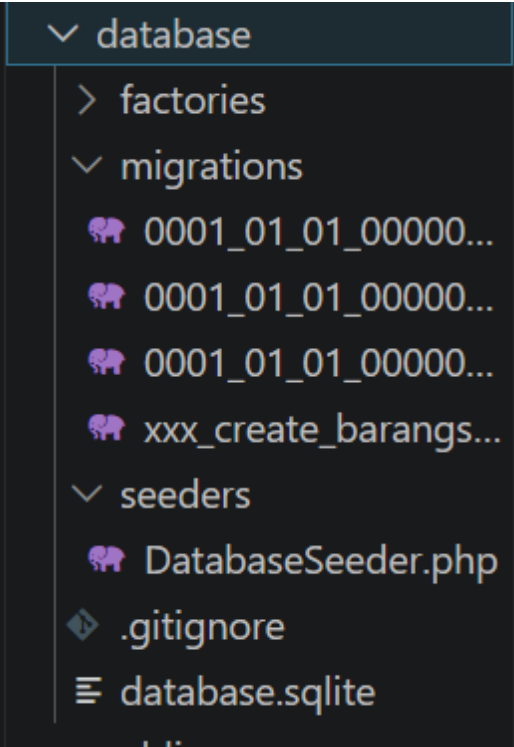
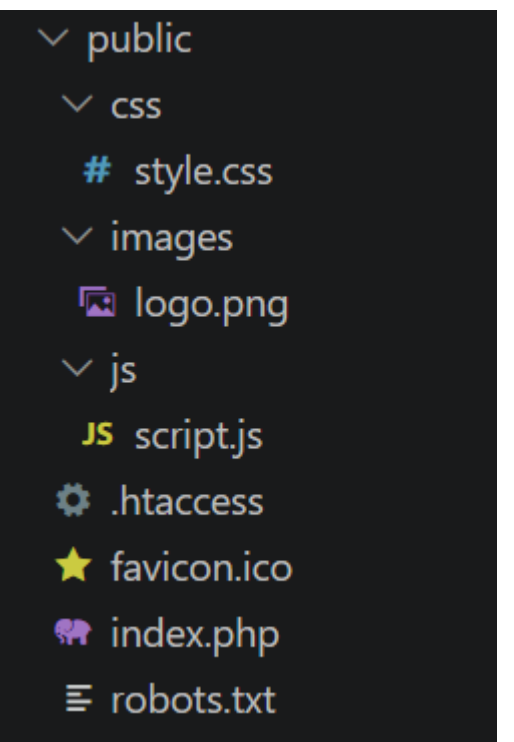
Foto Folder	Deskripsi
-------------	-----------

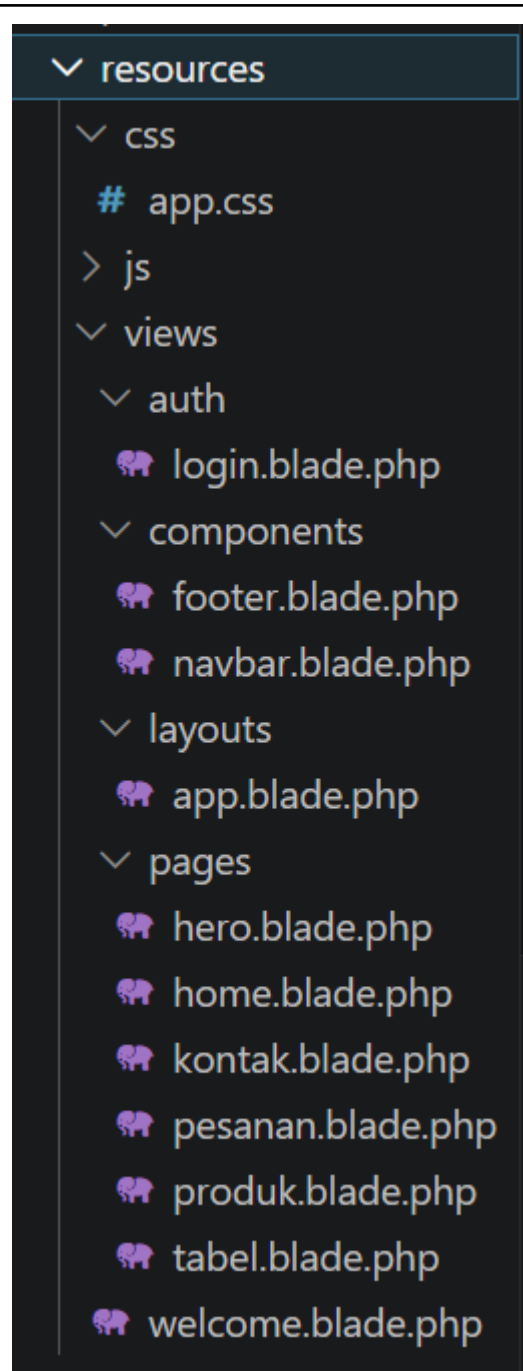
	Folder ini merupakan inti dari aplikasi karena berisi kode utama yang mengatur jalannya sistem. Di sinilah proses bisnis dan pengolahan data dilakukan. ● Http/Controllers/ Berisi controller yang berfungsi sebagai penghubung antara permintaan dari pengguna (request) dengan respon yang diberikan oleh sistem..
	Folder ini menjadi pusat dari seluruh logika aplikasi. Semua proses utama seperti pengolahan data dan alur program dikelola di dalam direktori ini. ● Http/Controllers/ Berisi controller yang bertugas menangani request dari pengguna dan mengatur respon yang akan diberikan, biasanya dengan menghubungkan ke model atau view. ● Models/ Digunakan untuk merepresentasikan struktur data yang ada di database serta mengatur proses interaksi data seperti mengambil, menyimpan, dan mengubah data. ● Providers/ Folder ini berisi service provider yang berfungsi untuk melakukan inisialisasi berbagai layanan dalam aplikasi. Service provider ini biasanya digunakan untuk mendaftarkan komponen, binding, atau konfigurasi yang akan digunakan secara global oleh Laravel.



Folder ini berisi berbagai file konfigurasi yang digunakan untuk mengatur cara kerja aplikasi Laravel secara keseluruhan. Dari gambar terlihat beberapa file penting yang mengatur bagian inti sistem.

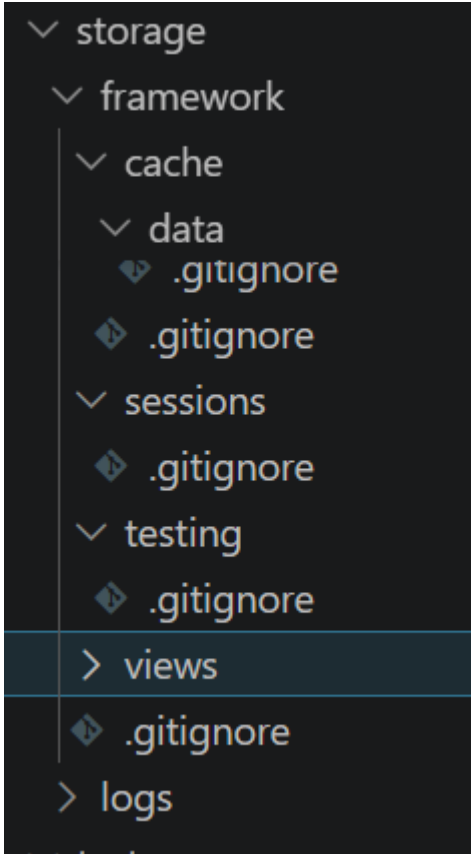
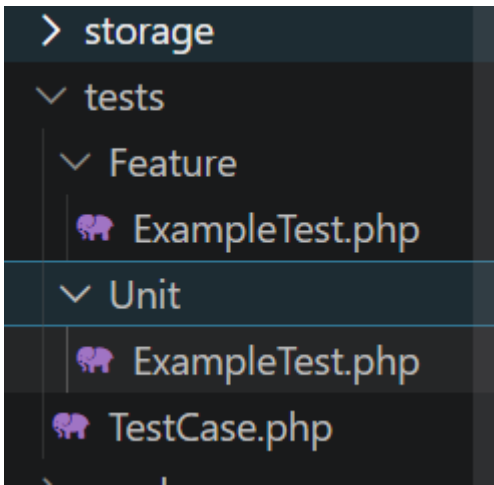
- **app.php**
Digunakan untuk mengatur identitas dan pengaturan dasar aplikasi seperti nama aplikasi, timezone, dan konfigurasi global lainnya.
- **database.php**
Berisi pengaturan koneksi database, seperti jenis database yang digunakan, host, username, dan password.
- **auth.php**
Mengatur sistem login dan autentikasi pengguna, termasuk bagaimana user diverifikasi.
- **session.php**
Digunakan untuk mengatur bagaimana data session disimpan dan digunakan selama pengguna mengakses aplikasi.

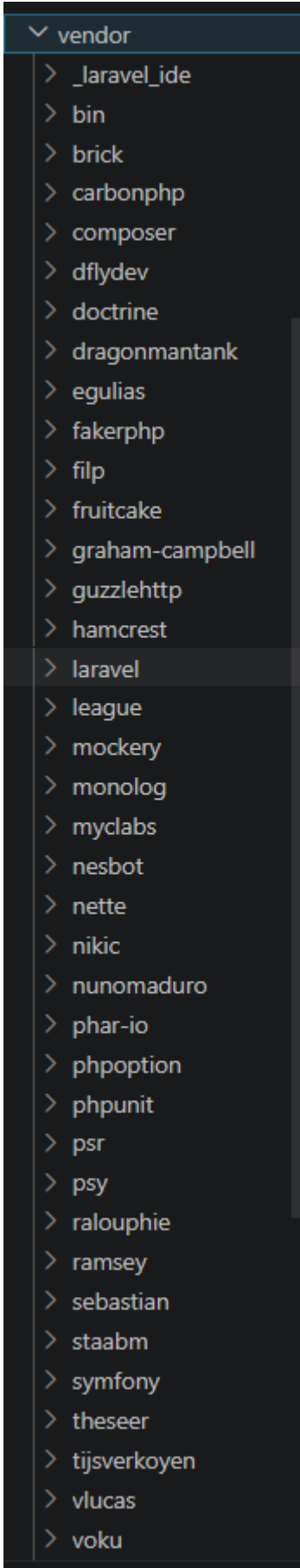
 <pre> database ├── factories ├── migrations │ ├── 0001_01_01_00000... │ ├── 0001_01_01_00000... │ ├── 0001_01_01_00000... │ └── xxx_create_barangs... ├── seeders │ └── DatabaseSeeder.php ├── .gitignore └── database.sqlite </pre>	folder ini memiliki beberapa bagian yang berhubungan langsung dengan pengelolaan database. • migrations/ Berisi file untuk membuat struktur tabel database, misalnya tabel produk atau transaksi. • factories/ Digunakan untuk membuat data dummy secara otomatis, biasanya untuk testing. • seeders/ Digunakan untuk mengisi data awal ke dalam database agar tidak kosong saat pertama dijalankan.
 <pre> public ├── css │ └── style.css ├── images │ └── logo.png ├── js │ └── script.js ├── .htaccess ├── favicon.ico ├── index.php └── robots.txt </pre>	Folder ini adalah bagian yang langsung terhubung dengan browser. • css/ Berisi file styling untuk tampilan website. • js/ Berisi script JavaScript untuk interaksi pada halaman. • images/ Digunakan untuk menyimpan gambar seperti logo atau gambar produk (sesuai yang terlihat di gambar PDF kamu).



Dari gambar terlihat folder ini digunakan untuk menyimpan file tampilan.

- **views/**
Berisi file tampilan dengan format .blade.php. Biasanya di dalamnya ada pembagian seperti layout utama dan komponen (misalnya navbar, footer).
- **css/**
Tempat menyimpan file CSS sebelum diproses lebih lanjut.
- **js/**
Tempat menyimpan file JavaScript sebelum diproses.

	Folder ini digunakan untuk menyimpan berbagai data yang dihasilkan oleh aplikasi. • app/ Menyimpan file yang diupload oleh user. • framework/ Berisi data sementara seperti cache dan session. • logs/ Berisi file log yang mencatat aktivitas dan error aplikasi.
	folder ini digunakan untuk testing aplikasi. • Feature/ Digunakan untuk menguji fitur secara keseluruhan. • Unit/ Digunakan untuk menguji bagian kecil dari kode

 ▼ vendor > _laravel_ide > bin > brick > carbonphp > composer > dflydev > doctrine > dragonmantank > egulias > fakerphp > filp > fruitcake > graham-campbell > guzzlehttp > hamcrest > laravel > league > mockery > monolog > myclabs > nesbot > nette > nikic > nunomaduro > phar-io > phoption > phpunit > psr > psy > ralouphie > ramsey > sebastian > staabm > symfony > theseer > tijsverkoyen > vlucas > voku	Folder ini berisi semua library yang diinstall melalui Composer (misalnya Laravel, Symfony, dll). Folder ini tidak boleh diedit secara manual karena berisi file sistem dari pihak ketiga
--	---

```

≡ .editorconfig
⚙ .env
⚙ .env.example
🔍 .gitattributes
🔍 .gitignore
npm .npmrc
≡ artisan
{} composer.json
{} composer.lock
{} package.json
📡 phpunit.xml
📖 README.md
⚡ vite.config.js

```

folder ini terdapat beberapa file penting:

- **.env**
Berisi konfigurasi penting seperti database dan key aplikasi.
- **artisan**
Digunakan untuk menjalankan perintah Laravel melalui terminal.
- **composer.json**
Berisi daftar dependency PHP.
- **package.json**
Berisi dependency JavaScript.
- **vite.config.js**
Digunakan untuk konfigurasi build frontend.

9. Jalankan php artisan route:list dan catat route apa saja yang sudah terdaftar

```

PS C:\xampp\htdocs\BioAquaLab> php artisan route:list

GET|HEAD / ..... HomeController@index
GET|HEAD login ..... AuthController@login
POST login ..... AuthController@authenticate
POST pesanan ..... PesananController@store
GET|HEAD storage/{path} storage.local > vendor/laravel/framework/src/Illuminate/Filesystem/Fil...
PUT storage/{path} storage.local.upload > vendor/laravel/framework/src/Illuminate/Filesys...
GET|HEAD up vendor/laravel/framework/src/Illuminate/Foundation/Configuration/ApplicationBuilde...

Showing [7] routes

PS C:\xampp\htdocs\BioAquaLab>

```

Langkah terakhir pada aktivitas pertama adalah melakukan audit terhadap seluruh jalur navigasi yang terdaftar menggunakan perintah `php artisan route:list`. Melalui perintah ini, pengembang dapat memverifikasi endpoint URL yang tersedia, memastikan metode HTTP yang digunakan sudah tepat, serta memvalidasi keterhubungan antara URL dengan fungsi-fungsi pada controller terkait.

Aktivitas Mandiri 2

1. Buat Controller DashboardController: `php artisan make:controller DashboardController`

```
PS C:\xampp\htdocs\BioAquaLab> php artisan make:Controller DashboardController

[INFO] Controller [C:\xampp\htdocs\BioAquaLab\app\Http\Controllers\DashboardController.php] created successfully.

PS C:\xampp\htdocs\BioAquaLab>
```

Pada tahap ini, dibuat sebuah controller baru menggunakan perintah di terminal. Controller ini nantinya digunakan untuk mengatur logika pada halaman dashboard.

Setelah perintah dijalankan, sistem otomatis membuat file controller di dalam folder `app/Http/Controllers`. File ini akan menjadi penghubung antara data dan tampilan yang akan ditampilkan ke pengguna.

2. Tambahkan method `index()` yang mengembalikan `view('dashboard')` di DashboardController

```
app > Http > Controllers > DashboardController.php > DashboardController

1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Illuminate\Http\Request;
6
7  class DashboardController extends Controller
8  {
9      public function index()
10     {
11         return view('dashboard');
12     }
13 }
14
```

Di dalam controller yang sudah dibuat, ditambahkan sebuah method bernama `index()`. Method ini berfungsi untuk menangani permintaan ketika halaman dashboard diakses.

Berdasarkan yang terlihat pada gambar, method ini mengarahkan ke sebuah tampilan (view) bernama `dashboard`. Selain itu, pada tahap awal juga bisa ditambahkan data sederhana (sementara) untuk ditampilkan di halaman tersebut.

3. Daftarkan route di routes/web.php: `Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard')`

```
routes > web.php
1  <?php
2  use App\Http\Controllers\DashboardController;
3  use Illuminate\Support\Facades\Route;
4  use App\Http\Controllers\HomeController;
5  use App\Http\Controllers\PesananController;
6  use App\Http\Controllers\AuthController;
7
8  Route::get('/', [HomeController::class, 'index']);
9  Route::post('/pesanan', [PesananController::class, 'store']);
10
11 Route::get('/login', [AuthController::class, 'login']);
12 Route::post('/login', [AuthController::class, 'authenticate']);
13 Route::get('/dashboard', [DashboardController::class, 'index']);
14
```

Selanjutnya dilakukan pengaturan route pada file routes/web.php. Route ini berfungsi untuk menghubungkan URL dengan controller. Dari gambar, terlihat bahwa URL /dashboard diarahkan ke method index() yang ada di controller. Dengan adanya route ini, halaman dashboard bisa diakses melalui browser. Selain itu, biasanya route juga dikelompokkan untuk beberapa fitur lain agar lebih rapi dan mudah dikelola.

4. Buat file view sementara resources/views/dashboard.blade.php berisi '`<h1>Dashboard SIAK</h1>`'

```
<div class="hero-content">
  <p class="hero-tagline">♦ Sistem Manajemen Air Minum</p>
  <h1 class="hero-title">
    Selamat Datang di<br />
    <span>BioAqua Lab</span>
  </h1>Dashboard BioAqua Lab</h1>
  <p class="hero-desc">
    Air Minum Isi Ulang Kepercayaan Kita – Kelola stok, pemesanan, dan
    distribusi air mineral berkualitas dengan mudah dan efisien.
  </p>
```

Kemudian dibuat file tampilan di dalam folder resources/views dengan nama dashboard.blade.php.

Berdasarkan gambar, isi tampilan masih sederhana, yaitu hanya menampilkan judul halaman dashboard menggunakan tag heading. Ini menunjukkan bahwa halaman sudah berhasil terhubung dengan controller.

5. Akses `http://127.0.0.1:8000/dashboard` di browser — pastikan tampil



Setelah semua konfigurasi selesai, halaman dashboard diuji dengan membuka browser dan mengakses URL `/dashboard`.

Dari hasil yang terlihat pada gambar, halaman berhasil ditampilkan sesuai dengan isi file view. Hal ini menandakan bahwa route, controller, dan view sudah terhubung dengan benar.

6. Buat route `/tentang` dengan `Route::view('/tentang', 'tentang')` dan buat view-nya

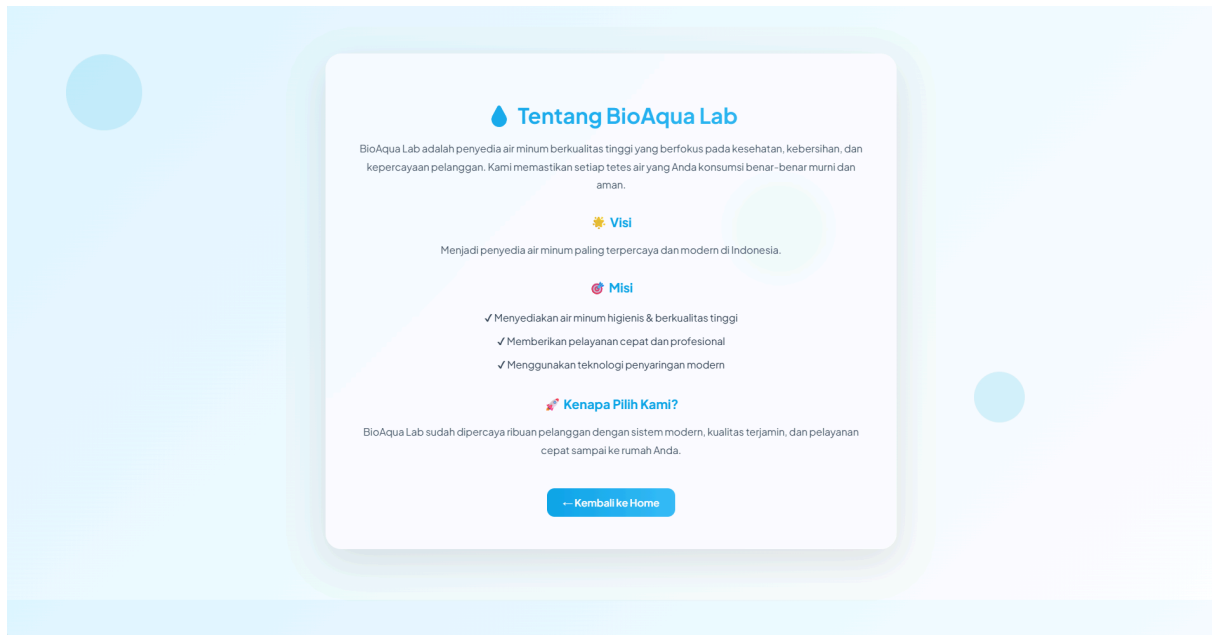

```

resources > views > tentang.blade.php > ...
1 <!DOCTYPE html>
2 <html lang="id">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Tentang Kami - BioAqua Lab</title>
7
8 <!-- Font modern -->
9 <link href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@400;500;600;700;800&display=swap" rel="stylesheet">
10
11 <style>
12 body{
13     margin:0;
14     font-family: 'Plus Jakarta Sans', sans-serif;
15     background: linear-gradient(135deg, #e0f7ff, #f0fbff, #ffffff);
16     color: #0f172a;
17 }
18
19 /* background bubble */
20 .bubble{
21     position:absolute;
22     border-radius:50%;
23     background: rgba(14,165,233,0.15);
24     animation: float 8s infinite ease-in-out;
25 }
26 .b1{width:120px;height:120px;top:10%;left:5%;}
27 .b2{width:80px;height:80px;top:60%;left:80%;}
28 .b3{width:150px;height:150px;top:30%;left:60%;}
29 @keyframes float{
30     0%,100%{transform:translateY(0);}
31     50%{transform:translateY(-20px);}
32 }
33
34 .container{
35     max-width:900px;
36     margin:auto;
37     padding:80px 20px;
38     position:relative;
39     z-index:2;
40 }
41
42 /* glass card */
43 .card{
44     background: rgba(255,255,255,0.7);
45     backdrop-filter:blur(15px);
46     border:1px solid rgba(255,255,255,0.4);
47     border-radius:25px;
48     padding:50px;
49     box-shadow:0 20px 60px rgba(0,0,0,0.1);
50     text-align:center;
51     animation: fadeIn 1s ease;
52 }
53
54 @keyframes fadeIn{
55     from{opacity:0; transform:translateY(20px);}
56     to{opacity:1; transform:translateY(0);}
57 }
58
59 h1{
60     font-size:34px;
61     margin-bottom:10px;

```

```
resources > views > ? tentang.blade.php > ...
2 <html lang="id">
3 <head>
11 <style>
89 .btn{
95     color: #ffffff;
96     border-radius:12px;
97     text-decoration:none;
98     font-weight:600;
99     transition:0.3s;
100 }
101 .btn:hover{
102     transform:scale(1.05);
103     box-shadow:0 10px 20px #rgba(14,165,233,0.3);
104 }
105 </style>
106 </head>
107 <body>
108 <div class="container">
109
110 <!-- bubble dekorasi -->
111 <div class="bubble b1"></div>
112 <div class="bubble b2"></div>
113 <div class="bubble b3"></div>
114
115 <div class="card">
116
117 <h1>💧 Tentang BioAqua Lab</h1>
118
119 <p>
120     BioAqua Lab adalah penyedia air minum berkualitas tinggi yang berfokus pada
121     kesehatan, kebersihan, dan kepercayaan pelanggan. Kami memastikan setiap tetes air
122     yang Anda konsumsi benar-benar murni dan aman.
123 </p>
124
125 <h2>🌟 Visi</h2>
126 <p>Menjadi penyedia air minum paling terpercaya dan modern di Indonesia.</p>
127
128 <h2>💖 Misi</h2>
129 <ul>
130 <li>✓ Menyediakan air minum higienis & berkualitas tinggi</li>
131 <li>✓ Memberikan pelayanan cepat dan profesional</li>
132 <li>✓ Menggunakan teknologi penyaringan modern</li>
133 </ul>
134
135 <h2>🔪 Kenapa Pilih Kami?</h2>
136 <p>
137     BioAqua Lab sudah dipercaya ribuan pelanggan dengan sistem modern,
138     kualitas terjamin, dan pelayanan cepat sampai ke rumah Anda.
139 </p>
140
141 <a href="/" class="btn">+ Kembali ke Home</a>
142
143 </div>
144 </div>
145 </body>
146 </html>
147
148
149
150
151
```





Selanjutnya dibuat halaman tambahan tanpa menggunakan controller, melainkan langsung menggunakan route sederhana.

Dari gambar terlihat bahwa route /tentang langsung diarahkan ke file view bernama tentang. Cara ini digunakan karena halaman tersebut hanya berisi tampilan statis tanpa proses logika.

Kemudian dibuat file tentang.blade.php yang berisi informasi mengenai aplikasi atau profil sesuai kebutuhan.

7. Jalankan php artisan route:list — amati semua route yang terdaftar

```
GET|HEAD / ..... HomeController@index
GET|HEAD dashboard ..... dashboard > DashboardController@index
GET|HEAD login ..... AuthController@login
POST login ..... AuthController@authenticate
POST pesanan ..... PesananController@store
GET|HEAD storage/{path} storage.local > vendor/laravel/framework/src/Illuminate/Filesystem/Fil...
PUT storage/{path} storage.local.upload > vendor/laravel/framework/src/Illuminate/Filesys...
GET|HEAD tentang .....
GET|HEAD up vendor/laravel/framework/src/Illuminate/Foundation/Configuration/ApplicationBuilde...

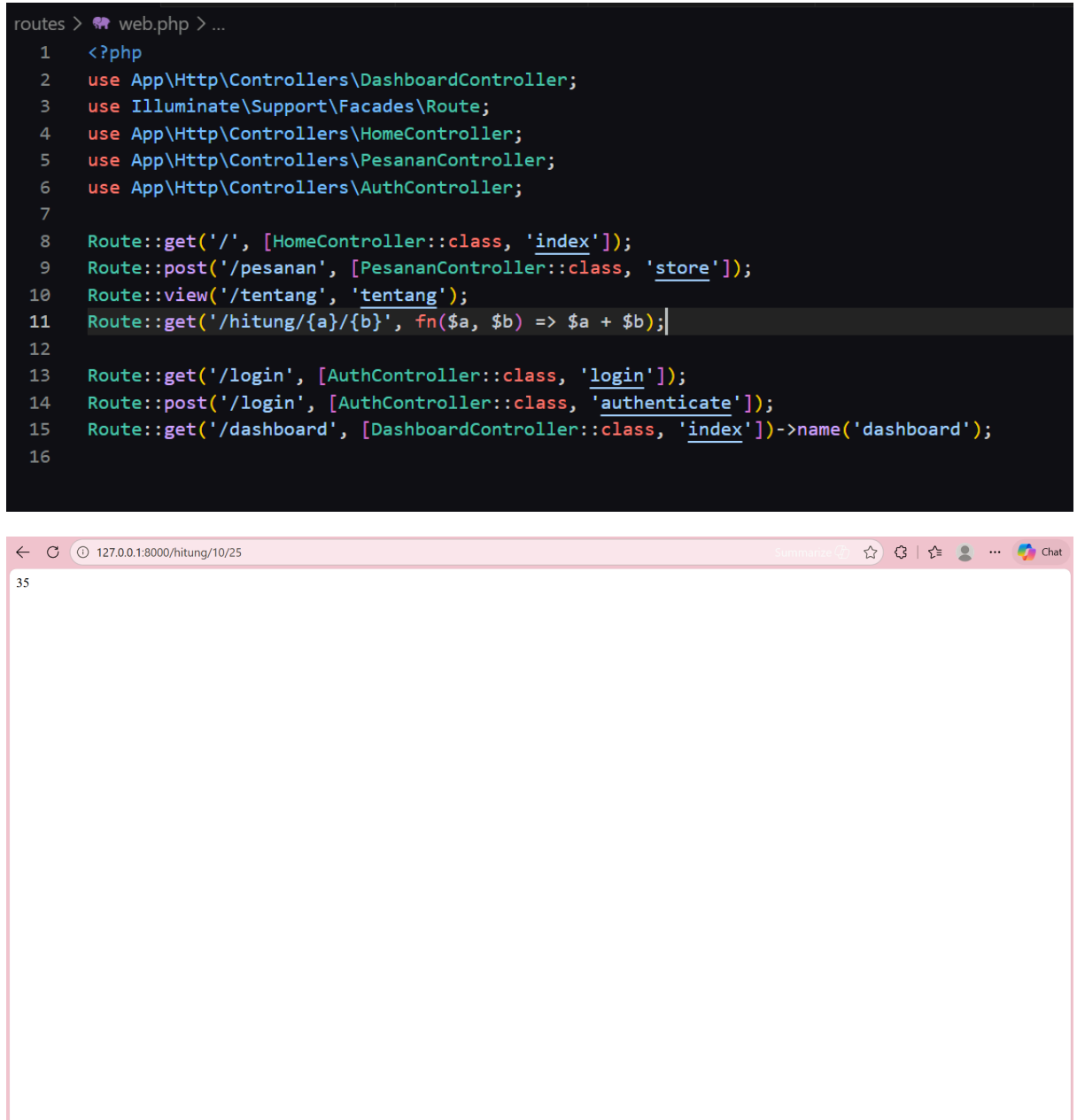
Showing [9] routes

PS C:\xampp\htdocs\BioAquaLab> |
```

Setelah menambahkan beberapa route, dilakukan pengecekan menggunakan perintah untuk menampilkan daftar route.

Dari hasil yang muncul (sesuai gambar), terlihat semua route yang sudah dibuat, termasuk route dashboard dan tentang. Ini digunakan untuk memastikan tidak ada kesalahan dalam penulisan route.

8. Coba buat `Route::get('/hitung/{a}/{b}', fn($a, $b) => $a + $b)` dan akses `/hitung/10/25`



Pada tahap terakhir, dibuat route yang dapat menerima input langsung dari URL.

Dari gambar terlihat bahwa route tersebut memiliki dua parameter. Ketika URL diakses dengan memasukkan dua angka, sistem akan memproses dan menampilkan hasilnya secara langsung.

Contoh yang ditunjukkan pada gambar adalah ketika dua angka dimasukkan, maka hasil penjumlahan langsung ditampilkan di browser. Ini menunjukkan bahwa Laravel dapat menangani input dinamis melalui URL.